

## Coding & Solution

|                      |                                                             |
|----------------------|-------------------------------------------------------------|
| <b>Date</b>          | 23 June 2026                                                |
| <b>Team ID</b>       | SWTID-2026-7169                                             |
| <b>Project Name</b>  | OptiCrop: Smart Agricultural Production Optimization Engine |
| <b>Maximum Marks</b> | 5 Marks                                                     |

### Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

#### Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

### Solution Summary

| Field                          | Details                                                                                                                                                                                               |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Repository Link / URL</b>   | <a href="https://github.com/Nazneen-1/OptiCrop-Smart-Agricultural-Production-Optimization-Engine.git">https://github.com/Nazneen-1/OptiCrop-Smart-Agricultural-Production-Optimization-Engine.git</a> |
| <b>Programming Language(s)</b> | Python                                                                                                                                                                                                |
| <b>Framework(s) Used</b>       | Flask (backend), HTML/CSS/Bootstrap (frontend), Scikit-learn (ML)                                                                                                                                     |

|                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Key Features Implemented</b>      | <ul style="list-style-type: none"> <li>• Data preprocessing (null check, outlier handling via IQR + log transform, seasonal feature extraction)</li> <li>• Model training and comparison (K-Means Clustering, Logistic Regression)</li> <li>• Model evaluation (Accuracy, Precision, Recall, F1-Score, Confusion Matrix)</li> <li>• Best model saved via Pickle (model.pkl)</li> <li>• Flask web app with Home, About, and FindYourCrop pages</li> <li>• Real-time crop prediction based on user-submitted soil/climate parameters</li> </ul> |
| <b>Pending / Incomplete Features</b> | <ul style="list-style-type: none"> <li>• Cloud deployment (currently runs on local Flask development server)</li> <li>• User authentication/login (not in current scope)</li> <li>• Historical prediction tracking / user report storage</li> </ul>                                                                                                                                                                                                                                                                                           |
| <b>Setup / Run Instructions</b>      | <ol style="list-style-type: none"> <li>1. Clone the repository and navigate to the project folder</li> <li>2. Create a virtual environment and install dependencies: pip install -r requirements.txt</li> <li>3. Ensure model.pkl is present in the project directory</li> <li>4. Run the application: python app.py</li> <li>5. Open the browser and go to the local server URL shown in the terminal (e.g., http://127.0.0.1:5000/)</li> </ol>                                                                                              |



### Code Quality Checklist

| S.No | Criteria                                               | Status (Yes / No) |
|------|--------------------------------------------------------|-------------------|
| 1    | Code is modular and organized into functions / classes | Yes               |
| 2    | Meaningful variable and function names are used        | Yes               |
| 3    | Code includes comments / documentation where necessary | Yes               |
| 4    | Error handling is implemented for critical operations  | Partial           |
| 5    | The application runs without critical errors           | Yes               |

|   |                                                   |     |
|---|---------------------------------------------------|-----|
| 6 | Code is committed to a version control repository | Yes |
|---|---------------------------------------------------|-----|

**Additional Notes / Comments:**

The OptiCrop application successfully integrates a trained Logistic Regression model with a Flask web interface to deliver real-time crop recommendations based on soil nutrient (N, P, K) and climate (temperature, humidity, pH, rainfall) parameters. Core functionality is complete and tested; future iterations could add cloud deployment and expanded error handling for production readiness.